

TRINNITY VISERON SYSTEM STARTUP PITCH ENGLISH

Multi-Agent AI Operating System — Real autonomy
· Multi-platform

14/14 tests passing

5,000+ autonomous agents

Persistent STM/LTM memory

4 active evolution cycles

290+ AI providers

Web · Mobile · Desktop · CLI

n8n + Voice + WhatsApp

Multi-cloud deploy ready

DATA · 06/08/2026

www.trinnityviseronssystem.io

TVS v5.0

1

EXECUTIVE SUMMARY

Trinnity Viseron System (TVS) is a multi-agent AI operating system designed to operate with real autonomy, evolve continuously, and coordinate thousands of specialized agents toward complex goals. TVS is not a chatbot or a single model — it is a self-organizing digital organization that plans and executes.

Unlike market AIs that require prompts, monitoring, and human maintenance, TVS boots by itself, spawns its own agents, executes tasks, generates reports, runs backups, and never stops — even when errors occur, it logs them and keeps going.

Key system metrics

5,000+ Autonomous agents

14/14 Core tests green

290+ AI providers

4 Evolution cycles

958 Indexed skills

6 Delivery platforms

3 Languages (PT/EN/ES)

100% Execution autonomy

2

THE PROBLEM

Today's AIs produce conversations, not results. Companies trying to automate with agents face structural barriers that consume time and money.

Limitations of current solutions

Chatbots & LLMs

- % Only respond when asked
- % Need human prompts and tuning
- % Do not evolve on their own
- % Forget everything each session

Agent frameworks

- % Agents die on every execution
- % No persistent memory
- % Manual orchestration required
- % A crash means lost work

The cost of human oversight

60–80% of the budget for AI agent projects is spent on monitoring, fixing, and re-running. Companies buy 'AI' but still pay operators to operate the AI. TVS removes that cost: agents monitor, fix, and evolve themselves.

3

THE SOLUTION

TVS is an AI operating system that executes real work without human intervention. From first boot, it starts servers, loads thousands of agents, connects integrations, and arms evolution cycles that run forever.

Layered architecture

Presentation WebOS (browser desktop), REST API, Socket.IO, PDF reports, Mobile App, Electron

Integrations n8n, OmniRoute (290+ providers), OpenJarvis, Call System (Twilio), ASNO (WhatsApp / Home Assistant)

Superintelligence Multi-provider ensemble synthesis, HyperLearning, AutoEvolution

Core Engine 5,000+ agents, squads, STM/LTM memory, tools, command chain, tokenomics

Competitive advantages

%, Real autonomy: AutoPilot generates tasks and executes them alone — up to 2 per cycle

%, Living memory: automatic STM!LTM consolidation every 30 minutes

%, Continuous evolution: intelligence multiplied each learning cycle

%, Immortality: no error takes down the process — log and continue

%, Watchdogs: every integration restarts itself if the process dies

4

PRODUCT — WHAT ALREADY WORKS

Unlike many pitch decks, TVS is not a prototype: it is an executable system with automated tests, a reproducible build, and multiple ready distribution artifacts.

Technical verification (August 2026)

npm test

14/14 core tests passing

npm run lint

TypeScript compiles with no errors

npm start

Boots Dashboard + ReportServer + n8n + OmniRoute automatically

npm run build:android

Builds APK for Android (Expo/EAS)

npm run build:exe

Standalone Windows executable (no Node.js needed)

npm run build:electron

Desktop app (Portable + Installer)

Proven features

Autonomy & AI

- % 4 active autonomous cycles
- % Multi-provider SuperMind + SuperIntelligence
- % Model Router: local (Ollama) + cloud
- % STM/LTM memory persisted to disk

Integrations & Interfaces

- % OmniRoute Hub — 290+ AI providers
- % n8n Bridge — 5 workflow templates
- % Call System — AI voice (Twilio)
- % WebOS, REST API, PDF, Mobile, Terminal

5 AUTONOMY & EVOLUTION

The 4 autonomous cycles — the heart of the system

- % HyperLearning (30 min) — Multiplies intelligence; queries AI, synthesizes insights, writes reports
- % AutoEvolution (60 min) — Agents gain knowledge and new capabilities; crossover creates hybrids
- % AutoLearning (30 min) — Consolidates short-term into long-term memory and measures real knowledge
- % AutoPilot (30 min) — Scans the system, generates tasks, and executes them — improves itself

Real autonomy example

AutoPilot scans the system and finds inactive agents. It creates the task 'Reactivate inactive agents', generates a strategy, and executes it through the multi-agent orchestrator. No human has to ask. As autonomy grows, it creates specialized agents, explores new integrations, and does deep optimization.

Intelligence growth

- % 0 · 100 (baseline) — System boot
- % 12 · ~179 — ×1.79 in 6 hours
- % 48 · ~1,000 — ×10 in 24 hours
- % 144 · ~10,000 — ×100 in 72 hours

6 INTEGRATIONS & ECOSYSTEM

Each integration boots on its own and is watched by a watchdog: if the process dies, TVS restarts it. No human needs to manage it.

Integration modules

- % OmniRoute — Gateway with 290+ AI providers · Port 20128 · auto-start · restart on exit
- % OpenJarvis — Personal local AI (Stanford-style) · Auto-start · restart on exit
- % n8n — Workflow automation engine · Port 5678 · 5 templates · local fallback
- % Call System — AI voice calls (Twilio) · Voice agents + call tracking
- % ASNO — WhatsApp + Home Assistant assistant · Webhooks + JARVIS WhatsApp
- % Viseron Apps — App integrations engine · Integration & agent stats
- % ReportServer — PDF + executive reports · Port 3001 · /report/pdf

Servers that boot on their own

- % Dashboard WebOS · porta 3000 — Browser desktop OS + Socket.IO + REST API
- % ReportServer PDF · porta 3001 — Executive PDF reports
- % Standalone Web · porta 3000 — Standalone web mode with ContentAgent
- % Forge Server · porta 4000 — Self-managed git hosting
- % n8n · porta 5678 — Workflow engine (spawned by TVS)
- % OmniRoute · porta 20128 — Gateway for 290+ providers (spawned)

7 MARKET

The autonomous AI agent market is growing explosively. Companies spend billions on agent monitoring labor — TVS attacks exactly that cost by offering a system that operates alone and pays for itself by eliminating manual oversight.

Target segments

SMBs (SaaS)

- % Content, support and process automation
- % Hours to value, not weeks
- % Price: \$29–\$99/month per instance
- % Large and fragmented market

Enterprise (License)

- % On-demand agent squads
- % On-premise deploy with 99.9% SLA
- % Price: \$499–\$4,999/month
- % High ticket, longer cycle

Why now

- % AI agent wave — 12–24 month advantage window
- % System already working today, not a slide: green tests and distributable artifacts
- % Cost of running AI dropped: local models (Ollama) + on-demand cloud
- % Full stack with multi-cloud deploy already configured

8

BUSINESS MODEL

Revenue streams

SaaS subscription

- % TVS Core \$29/month — up to 100 agents
- % TVS Pro \$99/month — up to 500 agents
- % TVS Enterprise \$499/month — unlimited
- % Annual plans with discount

Complementary revenue

- % Enterprise on-premise licensing
- % Skills/agents marketplace (20% commission)
- % Managed consulting & deployment
- % \$TRIN/\$VSR tokens as usage currency

Value mechanics

Each customer gets a system that operates 24/7: content published, workflows executed, support processed, code generated. The marginal cost per agent is low (local models + smart routing), securing 70–85% gross margins in SaaS.

Revenue projection (conservative)

- % Year 1 (v6.0) · 150 clientes · \$9K MRR · \$108K ARR — SaaS early adopters
- % Year 2 (v6.1) · 1,200 clientes · \$72K MRR · \$864K ARR — + channels & partnerships
- % Year 3 (v7.0) · 4,000 clientes · \$240K MRR · \$2.9M ARR — + Enterprise & marketplace

9

COMPETITION & DIFFERENTIATORS

Positioning

- % Chatbots (ChatGPT, Claude, Gemini) — On-demand conversations; No continuous execution; no long-term memory
- % Frameworks (LangChain, AutoGen) — Developer libraries; Requires heavy engineering; agents die per execution
- % Automation (n8n, Zapier, Make) — Visual workflows; No autonomous AI; manual configuration
- % Agents (Claude Agent, Manus) — Point assistance; No hierarchy/memory; high per-call cost

% Trinnity Viseron (TVS) — Complete system; Memory + hierarchy + self-healing + open-source

Defensible advantages (moat)

% Open-source with proprietary architecture — independent of any single framework

% Low operating cost: local models + routing across 290+ providers

% Ecosystem: 958 skills, n8n templates, planned marketplace

% Community and \$TRIN/\$VSR tokens as governance currency

10

ROADMAP

v5.0 (current) — Proven autonomy

% 5,000+ autonomous agents + 4 active evolution cycles

% AutoPilot executes tasks alone; autonomy up to 100%

% Standalone .exe working (no Node.js needed)

% WebOS, PT/EN/ES voice, n8n, tokens, mobile

v5.1 — Autonomy expansion

% Multi-tenant auth and billing (Stripe)

% Visual workflow editor in WebOS

% Drag-and-drop squad builder

% More languages: FR, DE, IT, JP, ZH

v6.0 — Decentralized network

% P2P agent network (multi-node cluster)

% Blockchain for tokens

% Third-party plugin marketplace

% Visual agent behavior editor

11

INVESTMENT REQUEST

TVS v5.0 already works end-to-end: it boots alone, evolves alone, and recovers alone. We are seeking investors to scale: multi-node cloud, p2p network, marketplace, and commercial growth.

Use of funds

50% Cloud infrastructure + multi-node cluster

25% Product: marketplace, visual editor, native app

15% Marketing and enterprise channels

10% Operations, support and compliance

Opportunity

% Working system today — not a prototype or slide deck

% Proven autonomy: 5,000+ agents, continuous evolution, self-healing

- %, Full stack (WebOS, voice, n8n, tokens, standalone exe, mobile)
- %, First-mover in 'AI that works alone' — no direct niche competition

12 CONTACT

LET'S BUILD THE FUTURE

Trinnity Viseron System v5.0

Multi-Agent AI Superintelligence

%, Ø=ÜQ Pedro Costa — Supreme Commander

%, Ø=Üx Trinnity Hurtado — Queen & Chief Architect

Contact: pedro@trinnity.com · trinnity@viseron.io

GitHub: github.com/ViseronSystem/trinnity-viseron-system

Dashboard: <http://localhost:3000>

© 2026 Trinnity Viseron System — All rights reserved